

Gateway — arbiter, actuation, backhaul

Owner: *TBD*

Depends on: `docs/PROTOCOL.md`, `firmware/` mesh, `backend/arbiter.py`

Deliverable for M4: a physical relay that clicks from a real quorum event, with measured latency under 2 s.

You own the only safety-critical software in the project. Everything else can fail gracefully; a bug here de-energises a live feeder or fails to de-energise a dangerous one. Write accordingly.

1. Position in the system

The gateway sits at the feeder head or substation. It:

1. Receives LoRa telemetry and votes from all nodes on the feeder
2. Runs `decide()` **locally**
3. Drives the isolation actuator
4. Forwards everything to the cloud over LTE, buffered when offline

The trip path never touches the internet. LTE round-trip plus cloud processing can exceed the entire 2 s budget on its own, and a safety function that depends on a mobile network is not something a utility will deploy. The cloud observes and audits. State this clearly whenever the architecture is questioned — it is one of the strongest engineering answers in the project.

2. Hardware

Part	Choice
Compute	Raspberry Pi Zero 2 W, or ESP32-S3 for a fully embedded build
Concentrator	SX1302 8-channel, or SX1262 single-channel for the prototype
Backhaul	SIM7600 LTE module
Actuator drive	opto-isolated relay board → contactor coil, or motorised load-break switch
Local UI	two LEDs (armed / fault) and a physical reset button

Part	Choice
Power	fed from the LV supply, with a supercap or small battery to survive the isolation it just commanded

That last row matters and is easy to miss: if the gateway is powered from the feeder it just isolated, it dies mid-operation. Budget for ride-through.

3. Structure

```

gateway/
├── src/
│   ├── main.py / main.c
│   ├── lora_rx.c          concentrator
│                           interface, packet auth check
│   ├── arbiter.c          SAME port as
│                           firmware/arbiter.c – shared file
│   ├── actuator.c         relay drive,
│                           interlocks, lockout state
│   ├── safety.c           watchdog, fail-
│                           safe rules, rate limiting
│   ├── uplink.py          MQTT over LTE,
│                           store-and-forward buffer
│   └── beacon.c           time-sync beacon
│                           for node slotting
└── test/                  HIL tests against
docs/vectors/

```

`arbiter.c` is shared with `firmware/`, not copied.
One implementation, one set of vectors, one place to fix a bug.

4. Fail-safe rules

These are requirements, not preferences. Each needs a test.

#	Rule	Rationale
1	Loss of LoRa comms → no trip , raise alarm	a jammed or failed radio must not de-energise a healthy feeder
2	Loss of LTE → keep operating, buffer uplink	cloud is not in the trip path
3	Gateway watchdog reset → come up in LOCKOUT , require manual arm	never auto-trip on a fresh, unvalidated state
4	Max one ISOLATE per feeder per 60 s	prevents oscillation
5	No auto-reclose, ever, in v1	reclosing onto a downed conductor is how people die

#	Rule	Rationale
6	Physical lockout switch overrides all software	linemen must be able to trust a mechanical interlock
7	Default mode ALERT_ONLY	auto-isolation is opted into per feeder by the utility

Rule 5 and rule 7 are what make this deployable rather than a science project. Lead with them.

5. Latency budget

Stage	Budget
Field collapse → node detector asserts SUSPECT	250 ms (5 × 50 ms windows)
SUSPECT broadcast → neighbour votes received	600 ms
Gateway <code>decide()</code>	< 10 ms
Relay/contactor mechanical operation	100–400 ms

Stage	Budget
Total	~1.3 s, margin to 2 s

Instrument every stage. `latency_ms` in the `command` message is measured from the earliest SUSPECT to command issue; the actuator's own confirmation timestamp is logged separately. When a judge asks "how do you know it is under 2 seconds," you show measurements, not a budget table.

6. Task list

Phase 1 — receive

- [] Concentrator up, receiving from three `firmware/` dev boards
- [] Packet auth verification (AES-CCM), drop unauthenticated frames and count them
- [] Time-sync beacon transmitted, nodes slotting correctly

Phase 2 — decide

- [] `arbiter.c` shared with firmware, HIL test against all vectors
- [] Per-feeder `ArbiterConfig` loaded from local file, overridable from cloud

- ☐ Latency instrumentation at every stage

Phase 3 – actuate → M4

- ☐ Opto-isolated relay drive, LED indication
- ☐ All seven fail-safe rules implemented, each with a test
- ☐ Physical lockout switch wired and honoured
- ☐ Measured end-to-end latency from real quorum to relay click

Phase 4 – uplink

- ☐ MQTT over LTE, TLS, client certificate
- ☐ Store-and-forward: survive 1 h offline, replay in order on reconnect
- ☐ Remote config pull, signed
- ☐ Heartbeat so the cloud can detect a dead gateway

Phase 5 – field readiness

- ☐ Power ride-through through a commanded isolation
- ☐ Enclosure, DIN rail mount
- ☐ Cold-start behaviour verified: comes up in LOCKOUT
- ☐ 24 h soak test with no false assertion

7. Acceptance criteria for M4

1. Physical break on the scaled rig → relay operates, measured latency logged and under 2 s.
2. Unplug the LoRa antenna → alarm raised, relay does **not** operate.
3. Power-cycle the gateway → comes up in LOCKOUT, requires manual arm.
4. Trigger two breaks 10 s apart → second is rate-limited and logged, not actioned.
5. Cloud offline for an hour → events buffered and replayed in correct order.

8. Gotchas

- **Contactor coils are inductive.** Flyback diode or you will destroy the driver and possibly the SBC.
- **Ground loops between the LV sense side and the SBC** will inject noise into everything. Opto-isolate every crossing.
- **Do not let the cloud become a control path by accident.** It is easy, under demo pressure, to wire a "trip" button in the dashboard straight through. If you add manual command from cloud, it must land as a *request* that the gateway independently validates, rate-limits and can refuse.
- Test the LTE path on the actual venue network before the finale. Campus wifi and a SIM behind

CGNAT behave differently.